Exercice 1 :

Que fait ce programme et quel est le résultat attendu après exécution

```
<?php
class BaseClass {
   function __construct() {
      print "In BaseClass constructor\n";
   }
}

class SubClass extends BaseClass {
   function __construct() {
      parent::__construct();
      print "In SubClass constructor\n";
   }
}
```

Exercice 2 : est ce que ce programme est correcte

```
class Utilisateur {
    // 1. Déclaration des propriétés
    public string $nom;
    private int $age;

    // 2. Initialisation via le constructeur
    public function __construct(string $nom, int $age) {
        $this->nom = $nom;
        $this->age = $age;
    }
}

// Instanciation
$user = new Utilisateur("Alice", 30);
```

Exercice 3 : est ce que ce programme est correcte

```
class Utilisateur {

    public function __construct(public string $nom, private int $age) { }
}
$user = new Utilisateur("Alice", 30);
```

Exercice 4 : est ce que ce programme est correcte si oui que fait t il

```
class Configuration {
    public function __construct(
        public readonly string $apiKey,
        public readonly string $environnement
    ) {}
}

$config = new Configuration("12345-XYZ", "production");
echo $config->apiKey;

$config->apiKey = "nouvelle-clef";
Configuration::$apiKey
```

Exercice 5 : est ce que le programme est correct et afficher le résultat

```
<?php
class PropertyTest
{
    private $data = array();
    public $declared = 1;
    private $hidden = 2;

    public function __set($name, $value)
    {
        echo "Setting '$name' to '$value'\n";
        $this->data[$name] = $value;
    }

    public function __get($name)
    {
        echo "Getting '$name'\n";
        if (array_key_exists($name, $this->data)) {
            return $this->data[$name];
        }
}

$obj = new PropertyTest;
$obj->a = 1;
echo $obj->a;
```

Exemple 6 : différenciant self et static : est ce que le programme est correct et afficher le résultat

```
class ParentClass {
  public static function quiSuisJe() {
    echo "Je suis ParentClass\n";
```

```
    }

    public static function tester() {
        self::quiSuisJe();
        static::quiSuisJe();
    }
}

class EnfantClass extends ParentClass {
    public static function quiSuisJe() {
        echo "Je suis EnfantClass\n";
    }
}

EnfantClass::tester();
```

Exemple 7 : d'utilisation des fonctions magiques set et get

```
class Utilisateur {
    private $donnees = [];

    public function __set($nom, $valeur) {
        echo "Assignation de '$valeur' à la propriété '$nom'\n";
        $this->donnees[$nom] = $valeur;
    }

    public function __get($nom) {
        echo "Lecture de la propriété '$nom'\n";
        if (array_key_exists($nom, $this->donnees)) {
            return $this->donnees[$nom];
        }
        return null;
    }
}

$user = new Utilisateur();
$user->prenom = "Alice";
echo $user->prenom;
```

En PHP, les méthodes magiques `__get` et `__set` permettent de lire et d'écrire dynamiquement des données dans des propriétés privées ou inexistantes d'un objet. Voici un exemple concret comparant l'utilisation de ces méthodes magiques avec une approche classique (sans méthodes magiques).

---

1. Avec l'utilisation de `__get` et `__set` (Méthodes magiques)

Dans cette version, nous utilisons un seul tableau privé `$data` pour stocker toutes nos propriétés. Les méthodes magiques interceptent l'accès aux propriétés comme si elles étaient publiques.

```
<?php

class UtilisateurMagique {
    // Tableau pour stocker dynamiquement les données
    private $data = [];

    // Déclenché lors de l'écriture : $objet->cle = $valeur
    public function __set($name, $value) {
        $this->data[$name] = $value;
    }

    // Déclenché lors de la lecture : echo $objet->cle
    public function __get($name) {
        if (array_key_exists($name, $this->data)) {
            return $this->data[$name];
        }
        return null; // Retourne null si la propriété n'existe pas
    }
}

// Utilisation
$user = new UtilisateurMagique();

// L'écriture appelle automatiquement __set()
$user->nom = "Alice";
$user->email = "alice@example.com";

// La lecture appelle automatiquement __get()
echo "Nom : " . $user->nom . "<br>";
echo "Email : " . $user->email . "<br>";
```

---

2. Le même code sans l'utilisation de `__get` et `__set` (Approche classique)

Pour reproduire exactement le même comportement (protéger l'accès direct aux variables), on définit explicitement des propriétés privées et on crée des méthodes "getters" et "setters" standards pour chacune d'elles.

php

```
<?php

class UtilisateurClassique {
    // Les propriétés sont définies explicitement de manière privée
    private $nom;
    private $email;

    // Getter et Setter pour le Nom
    public function setNom($nom) {
        $this->nom = $nom;
    }

    public function getNom() {
        return $this->nom;
    }

    // Getter et Setter pour l'Email
    public function setEmail($email) {
        $this->email = $email;
    }

    public function getEmail() {
        return $this->email;
    }
}

// Utilisation
$user = new UtilisateurClassique();

// L'écriture se fait via des méthodes explicites
$user->setNom("Alice");
$user->setEmail("alice@example.com");

// La lecture se fait via des méthodes explicites
echo "Nom : " . $user->getNom() . "<br>";
echo "Email : " . $user->getEmail() . "<br>";
```

🔄 Ce qui change en résumé

Critère	Avec `__get` / `__set` (Magique)	Sans `__get` / `__set` (Classique)
Syntaxe d'accès	`$user->nom = 'Alice';` (Plus propre/direct)	`$user->setNom('Alice');` (Plus verbeux)
Évolutivité	On peut ajouter n'importe quelle propriété à la volée.	Chaque nouvelle propriété nécessite d'écrire ses propres méthodes.
Auto-complétion (IDE)	Non supportée par défaut (nécessite des commentaires `@property`).	Parfaitement reconnue par les éditeurs de code.

Souhaitez-vous voir comment ajouter des **validations de données** (par exemple, vérifier si l'email est valide) à l'intérieur de la version **magique** ou de la version **classique** ?

Exercice 8 : que fait ce programme est ce que c'est correcte

```php
class Utilisateur implements Stringable {
    public function __construct(private string $nom) {}

    public function __toString(): string {
        return "Utilisateur: " . $this->nom;
    }
}

$user = new Utilisateur("Alice");
echo $user;
```

Exercice 9 : est ce que le programme ci-dessous est correct

```php
class Panier implements Countable {
    private array $articles = ['Pomme', 'Poire', 'Banane'];

    public function count(): int {
        return count($this->articles);
    }
}

$monPanier = new Panier();
echo count($monPanier);
```

Exercice 10 : est ce que le programme ci-dessous est correct

```
class Equipe implements IteratorAggregate {
    private array $joueurs = ["Léo", "Amine", "Sarah"];
    public function getIterator(): Traversable {

        return new ArrayIterator($this->joueurs);
    }
}

$equipe = new Equipe();
foreach ($equipe as $joueur) {
    echo $joueur . " ";
}
```

Exercice 11 : Expliquer les interfaces Countable et Stringable et IteratorAggregate